FACULTY OF COMPUTING

UNIVERSITI TEKNOLOGI MALAYSIA

DATA STRUCTURE AND ALGORITHM

SECJ2013-02

SEMESTER 1 2024/2025

ASSIGNMENT 1: ANALYSIS OF SORTING ALGORITHMS

LECTURER: DR ZURAINI BINTI ALI SHAH

GROUP 5

Name	Matric No.
BRENDAN CHIA YAN FEI (LEADER)	A23CS0211
GOH JIALE	A22EA0043
SALMAN SUHAIB MAJEED MAJEED	A22EC4013
YASMIN BATRISYIA BINTI ZAHIRUDDIN	A23CS0201

Table of Content

Introduction of Project

Based on the learning materials, sorting is a process in which data in a list is organized in a certain order, either in ascending or descending order. In our general understanding, a sorting algorithm is an algorithm that is used to sort a list of data into a well-structured way for better understanding the data. Sorting algorithms are divided into two types, which are quadratic sorting algorithms and divide and conquer sorting algorithms. Both types of them used to serve for different sizes of data. Quadratic sorting algorithms are more efficient in sorting small datasets or nearly sorted datasets. Divide-and-conquer sorting algorithms are suitable for both small and large datasets but may have overhead for very small datasets. In this project, we will explain the quadratic sorting algorithms, which are bubble sort, improved bubble sort, selection, and insertion sorting algorithms. The comparison performance, discussion of algorithm efficiency, and summary table will also be included in this report.

Bubble Sort: Code and Explanation

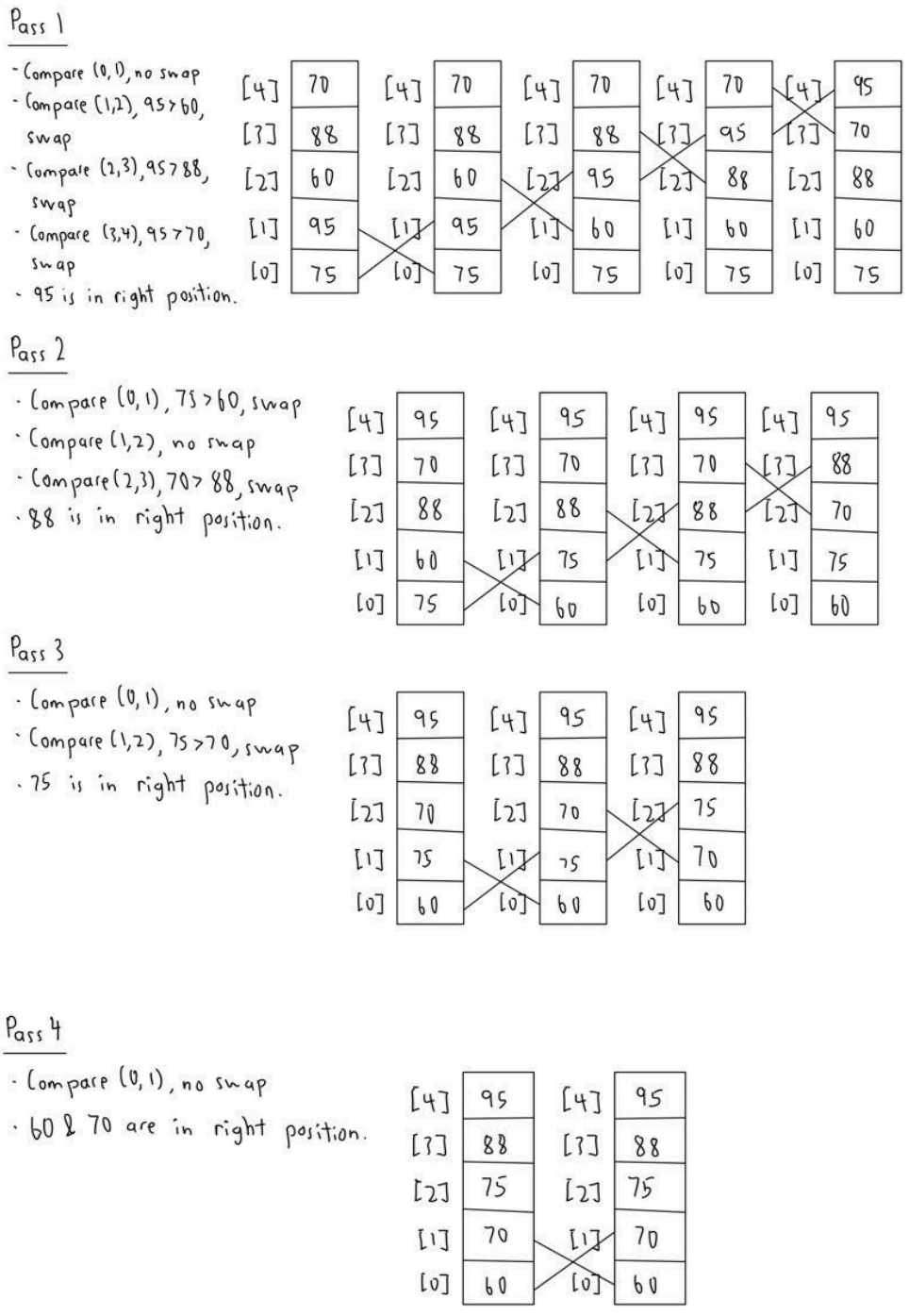
```
#include <iostream>
using namespace std;

void BubbleSort(int data[], int listSize){
  int pass, tempValue;
  int numberOfComparison = 0;
  int numberOfSwap = 0;
  for(pass = 1; pass < listSize; pass++){
    //moves the largest element to the end of array
    for(int x = 0; x < listSize - pass; x++){
      //compare adjacent elements
      numberOfComparison++;
      if(data[x] > data[x+1]){
        //swap elements
        tempValue = data[x];
        data[x] = data[x+1];
        data[x+1] = tempValue;
        numberOfSwap++;
      }
    }
  }
  cout << "Sorting in progress..." << endl;
  cout << "Number of Comparison is " << numberOfComparison << endl;
  cout << "Number of Swap is " << numberOfSwap << endl;
};

int main(){
  int data[] = {75, 95, 60, 88, 70};
  int n = sizeof(data)/sizeof(data[0]);

  cout << "Unsorted Array: ";
  for(int i = 0; i < n; i++){
    if (i < n-1){
      cout << data[i] << ",";
    } else {
      cout << data[i];
    }
```

```
  }
  cout << endl << endl;

  BubbleSort(data, n);
  cout << "Sorted Array: ";
  for(int i = 0; i < n; i++){
    if (i < n-1){
      cout << data[i] << ",";
    } else {
      cout << data[i];
    }
  }
  cout << endl;
}
```

Output:

```
Unsorted Array: 75,95,60,88,70

Sorting in progress...
Number of Comparison is 10
Number of Swap is 6
Sorted Array: 60,70,75,88,95
```

Explanation:



According to the figure above, the total number of swaps is 6.

Number of comparison = (5-4) + (5-3) + (5-2) + (5-1) = 10

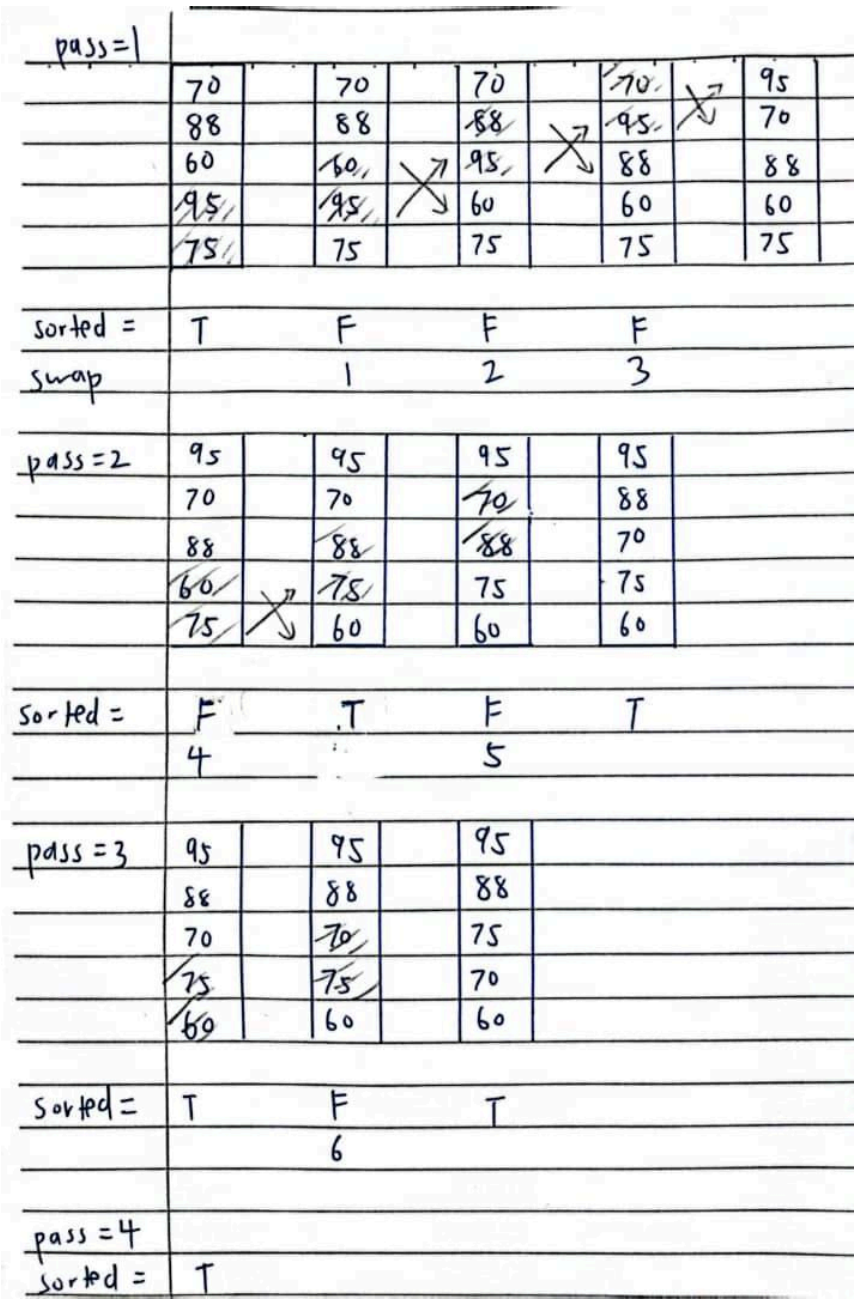
Improved Bubble Sort (with optimization): Code and Explanation

```
 #include<iostream>
using namespace std;


void bubbleSort(int data[], int n) {
    int temp;
    int numberOfComparison = 0;
    int numberOfSwap = 0;
    bool sorted = false; // false when swaps occur

    for (int pass = 1; (pass < n) && !sorted; ++pass) {
        sorted = true; // assume sorted
        numberOfComparison++;
        for (int x = 0; x < n - pass; ++x) {
            if (data[x] > data[x + 1]) {
                // exchange items
                temp = data[x];
                data[x] = data[x + 1];
                data[x + 1] = temp;
                sorted = false;
                numberOfSwap++;
               // signal exchange(if there is exchange of data)
            }
        }
    }


    // Another pass will continue if sorted is false and will stop if
sorted is true.
    cout << "Number of Comparison is " << numberOfComparison << endl;
    cout << "Number of Swap is " << numberOfSwap << endl;

} // end bubbleSort


int main(){
    int data[] = {75, 95, 60, 88, 70};
```

```
    int n = sizeof(data)/sizeof(data[0]);


    cout << "Unsorted Array: ";
    for(int i=0; i<n; i++){
        cout << data[i] << ", " ;
    }
    cout << endl;


    bubbleSort(data,n);
    cout << "Sorted Array: ";
    for(int i=0; i<n; i++){
        cout << data[i] << ", ";
    }


}
```

Output:

```
Unsorted Array: 75, 95, 60, 88, 70,
Number of Comparison is 4
Number of Swap is 6
Sorted Array: 60, 70, 75, 88, 95,
Press any key to continue . . .
```



According to the figure above, the total number of swaps is 6.

Number of comparison = 4

Selection Sort: Code and Explanation

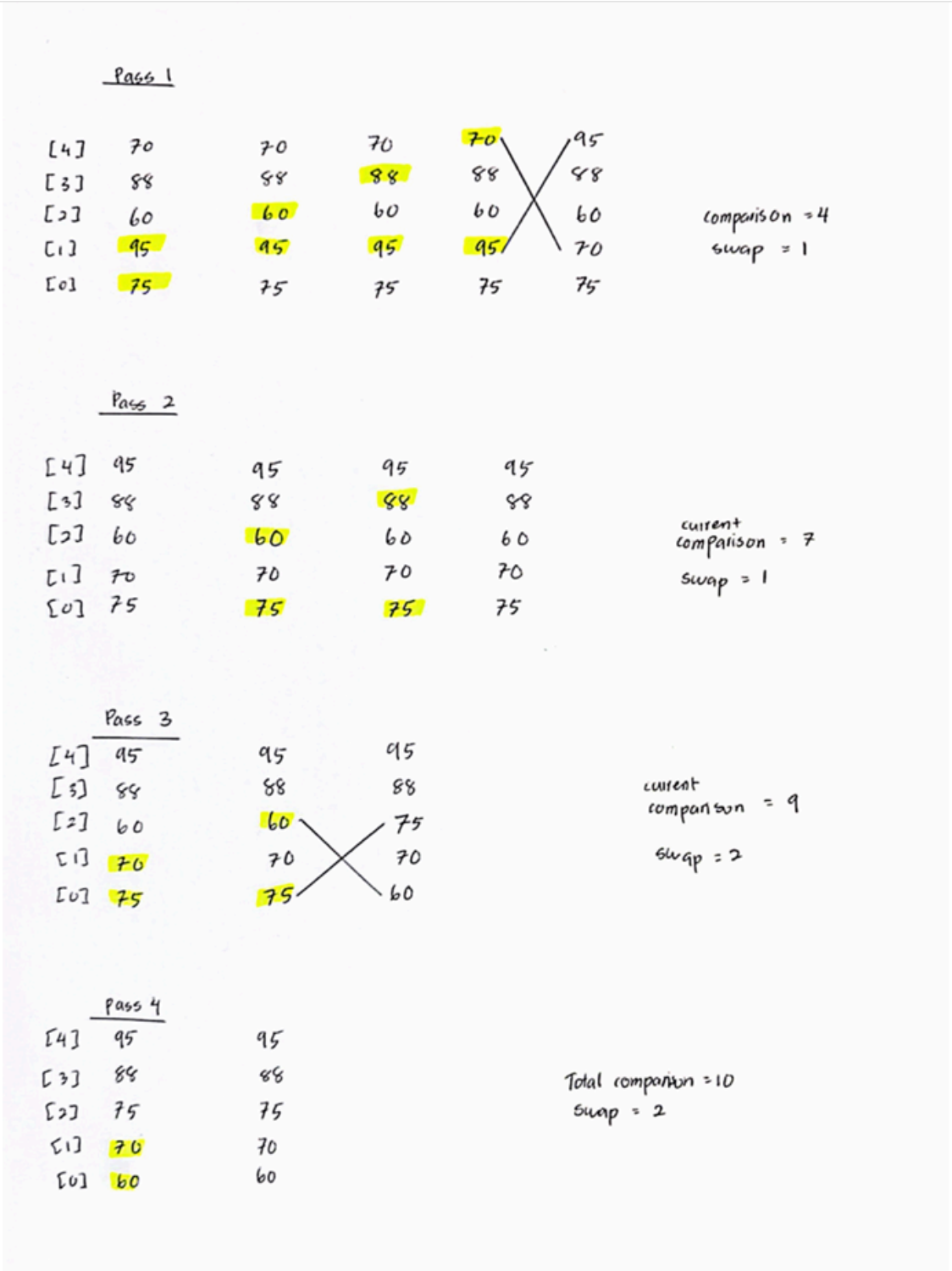
```
#include <iostream>
using namespace std;

void selectionSort(int Data[], int n, int &comparisonCount, int
&swapCount){

  for (int last = n-1; last >= 1; --last){
    int largestIndex = 0;

    for (int p=1;p <= last; ++p){
      comparisonCount++;
      if (Data[p] > Data[largestIndex])
      largestIndex = p;
    }
    if (largestIndex != last){
      swap(Data[largestIndex], Data[last]);
      swapCount++;
    }

  }
}


void swap(int& x, int& y){

  int temp = x;
  x = y;
  y = temp;

}


void printArray(int Data[], int n) {
    for (int i = 0; i < n; i++) {
        cout << Data[i] << " ";
    }
    cout << endl;
}

int main(){
```

```
  int Data[] = {75,95,60,88,70};
  int n = sizeof(Data) / sizeof(Data[0]);

  int comparisonCount = 0; // Initialize comparison counter
    int swapCount = 0;


  //print before sort
  cout << "Original Order: ";
    printArray(Data, n);

  selectionSort(Data, n, comparisonCount, swapCount); //sorting part

  //print after sort
  cout << "\nAfter Selection Sort: ";
    printArray(Data, n);

    // Print the counts
    cout << "\nTotal Comparisons: " << comparisonCount << endl;
    cout << "Total Swaps: " << swapCount << endl;

  return 0;


}
```

Output:

```
Original Order: 75 95 60 88 70

After Selection Sort: 60 70 75 88 95

Total Comparisons: 10
Total Swaps: 2

--------------------------------
Process exited after 1.644 seconds with return value 0
Press any key to continue . . .
```

Explanation:



Insertion Sort: Code and Explanation

```cpp
#include <iostream>
using namespace std;


void insertSort(int marks[], int n, int &compare, int &swap) {
    compare = 0; // To count the number of comparisons
    swap = 0;       // To count the number of swaps

    for (int i = 1; i < n; i++) {
        int key = marks[i];
        int j = i - 1;

        // Compare and shift elements
        while (j >= 0 && marks[j] > key) {
            compare++; // Increment comparisons
            marks[j + 1] = marks[j];
            j--;
            swap++; // Increment swaps as elements are shifted
        }

        marks[j + 1] = key; // Place the key in the correct position
        if (j >= 0) {
            compare++; // comparison where condition fails
        }
    }
}
```

```cpp
int main() {
    int marks[] = {75, 95, 60, 88, 70}; // array of student marks
    int n = sizeof(marks) / sizeof(marks[0]);
    int compare = 0, swap = 0;

    cout << "unsorted: ";
    for (int i = 0; i < n; i++) {
        cout << marks[i] << " ";
    }
    cout << endl;
    insertSort(marks, n, compare, swap);

    cout << "sorted: ";
    for (int i = 0; i < n; i++) {
        cout << marks[i] << " ";
    }
    cout << endl;

    cout << "Total comparisons: " << compare << endl;
    cout << "Total swaps: " << swap << endl;
    return 0;
}
```

Output :

```
unsorted: 75 95 60 88 70
sorted: 60 70 75 88 95
Total comparisons: 9
Total swaps: 6
```

Explanation:

Initialization:

- Comparisons and swaps variables are both initialized to zero to keep a count of the respective events in the algorithm.
- Array marks are iterated from the second element (i = 1) as it considers the first element to be sorted.

Key Selection:

- The key shall be the current element that is to be inserted into its correct position in the sorted part of the array.

Shifting Elements:

- Greater elements are shifted to the right side to make room for the key.
- Every shift is a swap, and the loop condition is a comparison, naturally enough.
- Insert: The key is inserted in its position when the loop terminates. If the condition happens to fail, then for completeness that is counted as a comparison. Output: The array is printed out along with the number of comparisons and swaps made in sorting.

Compare performance (number of comparisons and swaps) of each algorithm

For Bubble Sort, the number of comparisons is 10 and swaps is 6. Based on this observation, it has a higher number of swaps and comparison compared to other algorithms, indicating that it swaps more often during sorting.

For Improved Bubble Sort, it consists of 4 comparisons and 6 numbers of swaps. Although it consists of the same number of swaps as bubble sort, we notice that the number of comparisons significantly drops, indicating the improvement in performance.

For Selection Sort, it consists of 10 comparisons and 2 swaps, which actually is the lowest number of swaps. This finding shows that this algorithm minimizes unnecessary data movement but still compares at the same time with other algorithms.

Lastly, for Insertion Sort, it consists of 9 comparisons and 6 swaps. It performs fewer comparisons than Selection and Bubble Sort but still requires the same number of swaps as Bubble Sort.

Discussion of Efficiency of each algorithm based on your findings.

First and foremost, the Bubble Sort algorithm is the most inefficient algorithm compared to other algorithms. This is because it performs maximum comparisons and number of swaps, which indicates the slowest performance. Thus, it should rarely be used in practice due to its inefficiency.

Nonetheless, the Improved Bubble Sort algorithm has significantly improved in the number of comparisons, while the number of swaps still remains the same as Bubble Sort. This indicates that the algorithm can stop early if the array is sorted, but it still performs a large number of swaps. Since swapping is a waste of more time than comparing, it can only be considered moderate efficiency. Thus, it should be used in partial-sorted data structures.

Nevertheless, the Selection Sort algorithm is the most efficient algorithm in terms of swaps but still consists of a large number of comparisons. Since comparison is much cheaper than swap number in terms of the process time, it should be used when swapping operations are costly, such as when working with large datasets that are stored in external memory instead of internal.

Last but not least, the Insertion Sort algorithm is a balanced algorithm with a moderate number of comparisons and a large number of swaps. It should be less efficient compared to Improved Bubble Sort but still efficient compared to Bubble Sort. It is best for small or nearly sorted datasets due to its large number of swaps and lower comparison count compared to Bubble Sort.

Summary Table

Sorting Algorithm	Comparisons	Swaps
Bubble Sort	10	6
Improved Bubble Sort	4	6
Selection Sort	10	2
Insertion Sort	9	6